


# รายงานโครงงานการวิเคราะห์และออกแบบระบบ

# ระบบบริหารจัดการร้าน Minions Cafe (กลุ่มที่14)

## จัดทำโดย

67160002 นางสาวชาลิสา ศรีหงษ์

67160042 นางสาวมนทกานต์ ปาลี

67160178 นางสาวจตุพร ธรรมฤทธิ์

67160190 นางสาวบุษชยาภา คะสุดใจ

67160392 นางสาวชนิดาภา ชาภักดี

เสนอ

อาจารย์วิทวัส พันธุมจินดา

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 88734065 การวิเคราะห์และออกแบบระบบ

(System Analysis and Design)

คณะวิทยาการสารสนเทศ มหาวิทยาลัยบูรพา

| | รหัสนิสิต | ชื่อ - สกุล | บทบาทหน้าที่หลัก |
|---|---|---|---|
| 1 | 67160002 | นางสาวชาลิสา ศรีหงษ์ | Project Manager, System Analyst |
| 2 | 67160042 | นางสาวมนทกานต์ ปาลี | Project Lead Developer, System Analyst & Requirements Engineer |
| 3 | 67160178 | นางสาวจตุพร ธรรมฤทธิ์ | Business Analyst, Software Designer & UML Modeler |
| 4 | 67160190 | นางสาวบุษชยาภา คะสุดใจ | Database Designer, Backend Developer |
| 5 | 67160392 | นางสาวชนิดาภา ชาภักดี | UI/UX Designer, Frontend Developer & QA Specialist |

# สารบัญ

บทบาทหน้าที่หลัก ..... ก

สารบัญ ..... ข

1. บทนำ (Introduction) ..... 1

2. การวิเคราะห์ความเป็นไปได้ (Feasibility Study) ..... 1

3. การวิเคราะห์ความต้องการ (Requirements Analysis) ..... 3

4. User Stories และ Use Case ..... 7

5. การออกแบบเชิงวัตถุ (Object-Oriented Analysis & Design - OOAD) ..... 8

6. การออกแบบฐานข้อมูล (Database Design) ..... 10

7. การออกแบบกระบวนการและพฤติกรรมระบบ (Process & Behavioral Design) ..... 12

8. การออกแบบ Input/Output และส่วนประสานผู้ใช้ ..... 14

9. สถาปัตยกรรมระบบ (System Architecture - 4+1 View Model) ..... 16

10. ผลการทดสอบ Usability (Usability Testing) ..... 18

11. ผลการรวมระบบ (System Integration) ..... 19

12. ผลการตรวจสอบคุณภาพและความปลอดภัยของซอฟต์แวร์ ..... 20

13. สรุปผลและข้อเสนอแนะ (Conclusions & Recommendations) ..... 21

14. ภาคผนวก (Appendix: AI Disclosure Forms) ..... 22

# 1. บทนำ (Introduction)

**1.1 บริบทของร้านกาแฟ:** ร้านกาแฟ Minions Cafe เป็นธุรกิจร้านกาแฟสด ขนมอบ และเครื่องดื่มที่มีผู้เข้ามาใช้บริการอย่างต่อเนื่อง โดยมีกลุ่มลูกค้าเป้าหมายหลักคือนิสิต อาจารย์ และบุคลากรภายในมหาวิทยาลัย ตลอดจนประชาชนทั่วไปในพื้นที่ใกล้เคียง
ปัจจุบันร้านมีนโยบายในการขยายการดำเนินงานเป็นรูปแบบหลายสาขา (Multi-branch) ได้แก่ สาขา 1 (สาขาหลัก - ม.บูรพา) และ สาขา 2 (สาขาในเมือง - แหลมทอง)

**1.2 สภาพปัญหาเดิม:** จากการศึกษากระบวนการเดิมของร้านกาแฟ พบปัญหาคอขวดหลัก 4 ประการ ได้แก่ ความผิดพลาดในการจดบิลด้วยมือ (เฉลี่ยคิดเงินผิด 3-5 รายการต่อวัน),
ภาวะสต็อกวัตถุดิบหมดระหว่างวันโดยไม่รู้ตัวล่วงหน้า, การขาดข้อมูลยอดขายแบบเรียลไทม์ และความยุ่งยากในการแยกข้อมูลการขายข้ามสาขา

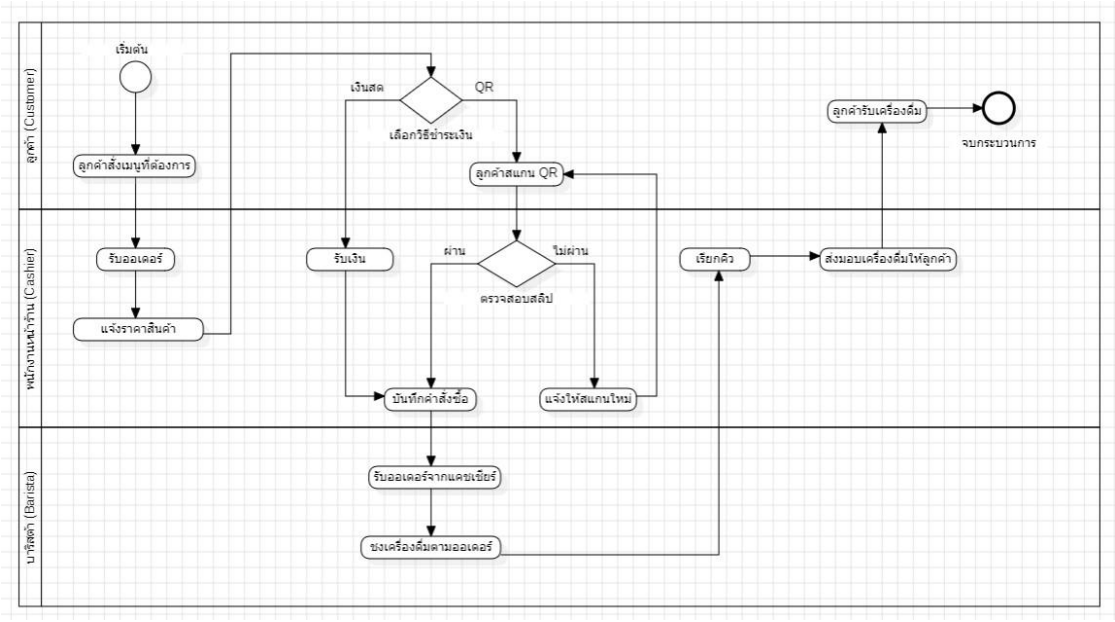
**1.3 วัตถุประสงค์ของโครงงาน:** โครงงานนี้มีวัตถุประสงค์เพื่อวิเคราะห์ ออกแบบ และพัฒนาระบบ POS หน้าร้านที่คำนวณเงินและออกใบเสร็จได้อย่างถูกต้องรวดเร็ว,
มีระบบตัดสต็อกอัตโนมัติพร้อมแจ้งเตือนสินค้าใกล้หมดและปิดปุ่มเมื่อสต็อกหมด (FR-11),
และมีแดชบอร์ดสรุปยอดขายแบบเรียลไทม์แยกสาขาตามหลัก 3NF และ 4+1 View Model

**1.4 องค์ประกอบ 6 ด้าน:** โครงงานครอบคลุมองค์ประกอบ 6 ด้านของระบบสารสนเทศ (People, Process, Data, Hardware, Software, Network) อย่างเป็นระบบตามที่ระบุใน Team Charter สัปดาห์ที่ 1

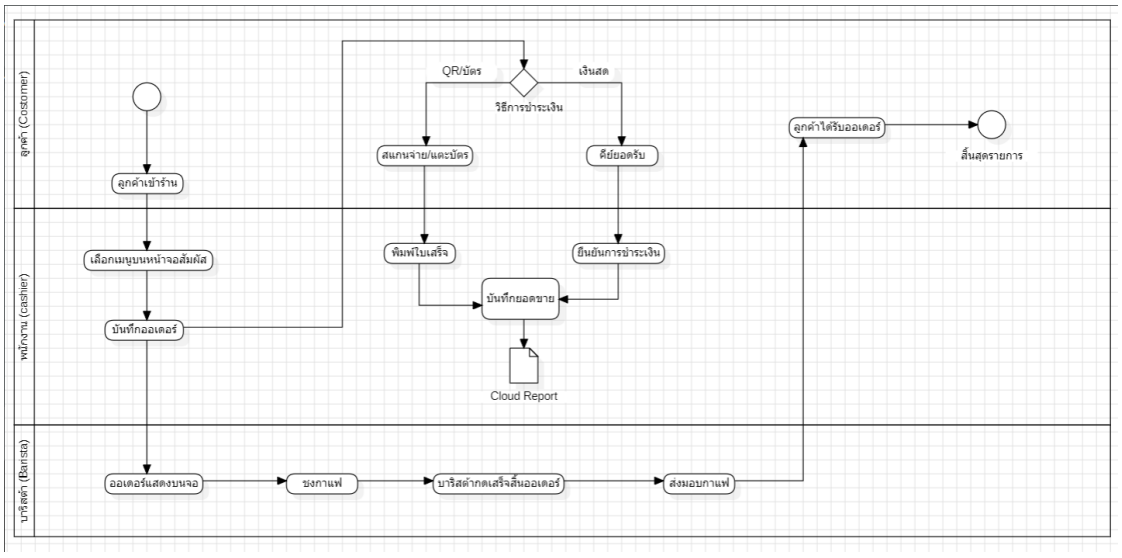
# 2. การวิเคราะห์ความเป็นไปได้ (Feasibility Study)

**2.1 การศึกษาความเป็นไปได้ (TELOS):** ทีมงานได้ศึกษาความเป็นไปได้ตามกรอบ TELOS (Technical, Economic, Legal, Operational, Schedule) พบว่ามีความคุ้มค่าสูง
สามารถลดความสูญเสียจากการคิดเงินผิดพลาดได้เดือนละกว่า 3,000 - 5,000 บาท
พนักงานเรียนรู้ระบบได้ภายใน 5 นาที และส่งมอบระบบได้ครบถ้วนภายใน 1 ภาคการศึกษา

## 2.2 การเปรียบเทียบกระบวนการทำงานเดิม (As-Is) และแบบใหม่ (To-Be)




*ภาพที่: แผนภาพกระบวนการทำงานเดิมก่อนมีระบบ (As-Is Process Diagram)*




*ภาพที่: แผนภาพกระบวนการทำงานใหม่หลังมีระบบ (To-Be Business Process Diagram)*

| กระบวนการ | เดิม (As-Is) | ใหม่ (To-Be) | ผลประโยชน์ที่ได้รับ |
|---|---|---|---|
| การรับออเดอร์ | จดลงสมุดกระดาษ ลายมืออ่านยาก ผิดพลาดบ่อย | กดเลือกเมนูผ่านหน้าจอสัมผัส POS | ลดข้อผิดพลาดเป็น 0% |
| การคิดเงิน/เงินทอน | กดเครื่องคิดเลขแยก คิดในใจ ช้าช่วงเร่งด่วน | ระบบคำนวณยอดรวมและเงินทอนอัตโนมัติสด | ปิดบิลเสร็จสิ้นใน 1-2 วินาที |
| การควบคุมสต็อก | ตรวจนับด้วยสายตา สต็อกหมดโดยไม่รู้ตัว | ตัดสต็อกอัตโนมัติ เตือนสต็อกต่ำ (<10) | วางแผนสต็อกล่วงหน้าได้แม่นยำ |
| การป้องกันข้อผิดพลาด | ไม่มีระบบตรวจสอบ รับเงินแล้วทำไม่ได้ | ปิดปุ่มเมนูที่สต็อกเป็น 0 ทันที (FR-11) | ป้องกันลูกค้ายกเลิกออเดอร์ |
| การออกใบเสร็จ | เขียนบิลเงินสดด้วยมือ หรือไม่ออกใบเสร็จ | สร้างใบเสร็จความร้อนมาตรฐานพร้อมเลขที่บิล | สร้างความน่าเชื่อถือและโปร่งใส |
| การสรุปยอดขาย | รวมบิลกระดาษตอนปิดร้าน เสียเวลามาก | แดชบอร์ดสรุปยอดขาย Real-time On-demand | เจ้าของร้านดูยอดและเมนูขายดีได้ทันที |

## 3. การวิเคราะห์ความต้องการ (Requirements Analysis)

**3.1 บันทึกการสัมภาษณ์ผู้ใช้จริง:** จากการสัมภาษณ์เจ้าของร้านและพนักงานแคชเชียร์ พบปัญหาใหญ่คือสต็อกวัตถุดิบชอบหมดกะทันหันในช่วงลูกค้าหนาแน่น และความต้องการทราบยอดขายทันทีโดยไม่ต้องรอบวกบิลตอนปิดร้าน งบประมาณไม่เกิน 60,000 บาท และระบบต้องไม่ซับซ้อนเพื่อให้พนักงานใหม่เรียนรู้ได้เร็ว

**3.2 ข้อกำหนดความต้องการเชิงหน้าที่ (Functional Requirements)**

- **FR-01:** ระบบต้องแสดงรายการเมนูตามสาขาที่เลือก โดยแบ่งตามหมวดหมู่ (กาแฟ, ชา, ขนม) พร้อมราคาและสต็อกคงเหลือ
- **FR-02:** ระบบต้องให้ผู้ใช้เลือกสินค้าและระบุจำนวนเข้าสู่ตะกร้าสินค้า และปรับเพิ่ม/ลดจำนวนหรือลบรายการได้ก่อนยืนยัน
- **FR-03:** ระบบต้องคำนวณราคารวมของแต่ละรายการและยอดรวมทั้งสิ้นแบบเรียลไทม์ทันทีที่มีการเปลี่ยนแปลง
- **FR-04:** ระบบต้องรองรับการเลือกวิธีชำระเงิน (เงินสด หรือ QR Code พร้อมเพย์) พร้อมคำนวณเงินทอนสำหรับเงินสด
- **FR-05:** ระบบต้องบันทึกคำสั่งซื้อ สร้างหมายเลขออเดอร์และออกใบเสร็จรับเงินดิจิทัลทันทีหลังยืนยันการชำระเงิน
- **FR-06:** ระบบต้องปิดการใช้งานปุ่มยืนยันออเดอร์หากไม่มีสินค้าในตะกร้า (Error Prevention)
- **FR-07:** ระบบต้องตัดสต็อกสินค้าในคลังทันทีเมื่อยืนยันคำสั่งซื้อ และบันทึกประวัติการเคลื่อนไหวสต็อก (Stock Movement)
- **FR-08:** ระบบต้องแสดงสัญลักษณ์แจ้งเตือน 'ต่ำกว่าเกณฑ์' (Low Stock) เมื่อสต็อกเหลือน้อยกว่า 10 หน่วย
- **FR-09:** ระบบต้องปฏิเสธการสั่งซื้อหากจำนวนสินค้าในคำสั่งซื้อมากกว่าสต็อกคงเหลือที่มีอยู่จริง
- **FR-10:** ระบบต้องรองรับการปรับปรุงสต็อกวัตถุดิบด่วน แก้ไขราคา และเพิ่มเมนูใหม่ประจำแต่ละสาขา
- **FR-11 (Feedback สำคัญ):** ระบบต้องปิดการใช้งานปุ่ม '+ เพิ่ม' (disabled) และแสดงสถานะ '✕ สินค้าหมด' สำหรับเมนูที่ stock_quantity = 0 ตั้งแต่หน้าแรก
- **FR-12:** ระบบต้องกรองการแสดงผลรายการเมนู สต็อก และรายงานยอดขายตาม branch_id ที่เลือกใน Header อย่างเคร่งครัด

**3.3 บันทึกประวัติการปรับปรุง Requirement:** จากการทดสอบ Prototype สัปดาห์ที่ 13 พบว่าหากไม่มีการปิดปุ่มตั้งแต่หน้าจอแรก ผู้ใช้จะยังกดเลือกสินค้าที่หมดได้ จึงเพิ่มข้อกำหนด FR-11 เพื่อป้องกันข้อผิดพลาดตั้งแต่ต้นทาง (Error Prevention) และปรับปรุง FR-12 ให้ตัวเลือกสาขาคงตำแหน่งมุมบนขวาในทุกหน้าจออย่างคงเส้นคงวา

**3.4 ข้อกำหนดความต้องการที่ไม่ใช่เชิงหน้าที่ (NFR):** ระบบกำหนด NFR-01 ด้านเวลาตอบสนอง (<100ms ในการคำนวณยอด, <1s ฝั่งเซิร์ฟเวอร์), NFR-02 ด้านความถูกต้องของสต็อก 100% ด้วย Database Transaction, NFR-03 ด้าน Usability พนักงานใหม่ใช้งานได้ใน 3 นาที และ NFR-04 ความคงเส้นคงวาของ Branch Selector ในทุกหน้าจอ

**3.5 ข้อจำกัดของระบบ (System Constraints)**

**ข้อจำกัดหลัก:** อ้างอิงจากเอกสารข้อกำหนดความต้องการซอฟต์แวร์ (wk10-srs.md) ระบบ Minions Cafe มีข้อจำกัดสำคัญ 5 ประการ:

- **1. ข้อจำกัดงบประมาณ (Budget):** งบประมาณจัดหาอุปกรณ์ POS, เครื่องพิมพ์ใบเสร็จความร้อน และการพัฒนาระบบรวมทั้งหมดไม่เกิน 60,000 บาท จึงเลือกใช้สถาปัตยกรรม Open Source (Node.js, Express, MySQL)
- **2. ข้อจำกัดฮาร์ดแวร์ (Hardware):** รองรับหน้าจอสัมผัส 16:9 สำหรับเคาน์เตอร์ POS และรองรับเครื่องพิมพ์ใบเสร็จความร้อนขนาด 80mm (@media print)
- **3. ข้อจำกัดความต่อเนื่อง (Fallback):** ระบบหน้าร้านต้องทำงานได้ต่อเนื่องแม้สัญญาณเน็ตขัดข้อง โดยรันผ่าน Local Express Server หรือ Standalone Web App
- **4. ข้อจำกัดการเรียนรู้ (Usability):** เนื่องจากพนักงานพาร์ตไทม์เปลี่ยนบ่อย ระบบต้องออกแบบให้เรียนรู้ขั้นตอนการขายได้ภายใน 5 นาทีโดยไม่ต้องมีคู่มือหนา
- **5. มาตรฐานธุรกรรมพาณิชย์ (Legal):** ใบเสร็จรับเงินต้องแสดงข้อมูลชื่อร้าน, สาขา, เลขที่ออเดอร์ (ORD-YYYYMMDD-XXX), วันเวลา, รายการ, และเงินทอนอย่างครบถ้วน

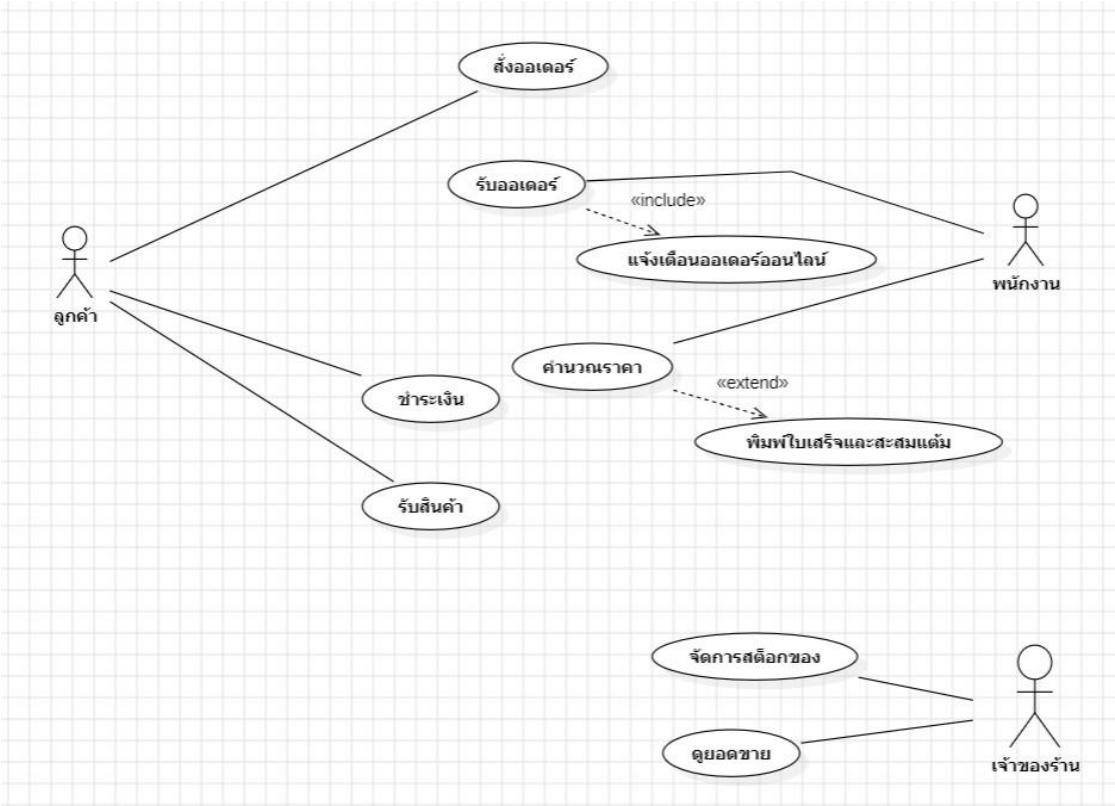
**3.6 ตารางความเชื่อมโยงความต้องการสู่การออกแบบและระบบจริง (Traceability Matrix)**

| ข้อกำหนด (FR) | สาระสำคัญของความต้องการ | Use Case | Domain Model / DB | Endpoint / ฟังก์ชันจริง |
|---|---|---|---|---|
| FR-01 | แสดงเมนู/สต็อกตามสาขา | UC-01 รับออเดอร์ | MenuItem / menu_item | GET /api/menu?branchId=:id |

| | | | | |
|---|---|---|---|---|
| FR-02 | ตะกร้าสินค้า ปรับจำนวน | UC-01 รับออเดอร์ | OrderItem / order_item | Frontend Cart State (app.js) |
| FR-03 | คำนวณยอดเงินสดเรียลไทม์ | UC-01 รับออเดอร์ | OrderItem.getSubtotal() | คำนวณราคาอัตโนมัติ (<100ms) |
| FR-04 | รับชำระเงินสด/QR คิดเงินทอน | UC-02 ชำระเงิน | Order.payment_method | คำนวณเงินทอนสดหน้าร้าน |
| FR-05 | ออกใบเสร็จรับเงินมาตรฐาน | UC-02 ชำระเงิน | Order, OrderItem / orders | Thermal Receipt Modal (80mm) |
| FR-07 | ตัดสต็อกอัตโนมัติ/Audit Trail | UC-03 ตัดสต็อก | MenuItem / stock_movement | POST /api/orders (ตัดยอดจริง) |
| FR-08 | แจ้งเตือนสต็อกต่ำ (<10) | UC-03 ตัดสต็อก | MenuItem.stock_quantity | ป้ายเตือนสีส้ม '⚠ เหลือ X ชิ้น' |
| FR-09 | ปฏิเสธการสั่งซื้อเกินสต็อก | UC-01, UC-03 | MenuItem.stock_quantity | Atomic Row Lock / 400 Bad Request |
| FR-11 | ปิดปุ่มเมนูสต็อก 0 ทันที | UC-01 รับออเดอร์ | MenuItem.stock_quantity | Disabled Button / '✕ สินค้าหมด' |
| FR-12 | แยกข้อมูลเมนู/ยอดตามสาขา | UC-04, UC-05 | branch_id (Foreign Key) | Branch Selector / ตรวจสอบสิทธิ์ IDOR |

# 4. User Stories และ Use Case

**4.1 ภาพรวม Actors และ Use Cases:** ระบบแบ่งผู้ใช้ออกเป็น 3 Actors หลัก ได้แก่ ลูกค้า (สั่งซื้อ/ชำระเงิน), พนักงานร้าน/แคชเชียร์ (รับออเดอร์/ตัดสต็อก/ออกใบเสร็จ), และเจ้าของร้าน/ผู้จัดการ (จัดการเมนู/ดูรายงานยอดขายเรียลไทม์)




*ภาพที่: Use Case Diagram ระบบบริหารจัดการร้าน Minions Cafe*

**4.2 User Stories และ Acceptance Criteria (Given-When-Then)**

**US-01: บันทึกออเดอร์หน้าร้าน:** ในฐานะพนักงานแคชเชียร์
ฉันต้องการเลือกเมนูเครื่องดื่มและระบุจำนวนผ่านหน้าจอ เพื่อให้ระบบคำนวณยอดรวมให้อัตโนมัติ

- **Acceptance Criteria:** Given อยู่หน้ารับออเดอร์, When เลือกเมนูและจำนวน, Then ระบบบันทึกรายการในตะกร้าและคำนวณยอดรวมสดทันที โดยสั่งซื้อไม่เกินสต็อกคงเหลือ

**US-02: ชำระเงินและออกใบเสร็จ:** ในฐานะลูกค้า ฉันต้องการเลือกชำระเงินด้วยเงินสดหรือ QR และได้รับใบเสร็จรับเงินทันที เพื่อตรวจสอบความถูกต้อง

- **Acceptance Criteria:** Given มีรายการในตะกร้า, When บันทึกการชำระเงินสำเร็จ, Then ระบบคำนวณเงินทอนและออกใบเสร็จดิจิทัลแสดงเลขที่ วันเวลา สาขา และรายการครบถ้วน

**US-03: การตัดสต็อกและเตือนสต็อกต่ำ:** ในฐานะพนักงานร้าน
ฉันต้องการให้ระบบตัดสต็อกอัตโนมัติและแจ้งเตือนเมื่อสต็อกต่ำ เพื่อเตรียมวัตถุดิบได้ทันการณ์

- **Acceptance Criteria:** Given ยืนยันออเดอร์สำเร็จ, When ระบบบันทึกรายการขาย, Then ระบบตัดสต็อกทันที หากสต็อกต่ำกว่า 10 จะขึ้นเตือนสีส้ม และหากสต็อกเป็น 0 จะปิดปุ่มเพิ่มสินค้า

**US-04: จัดการข้อมูลเมนูและสต็อก:** ในฐานะผู้จัดการร้าน ฉันต้องการเพิ่มเมนูใหม่ แก้ไขราคา และปรับปรุงสต็อก เพื่อให้ข้อมูลสอดคล้องกับการขายจริง

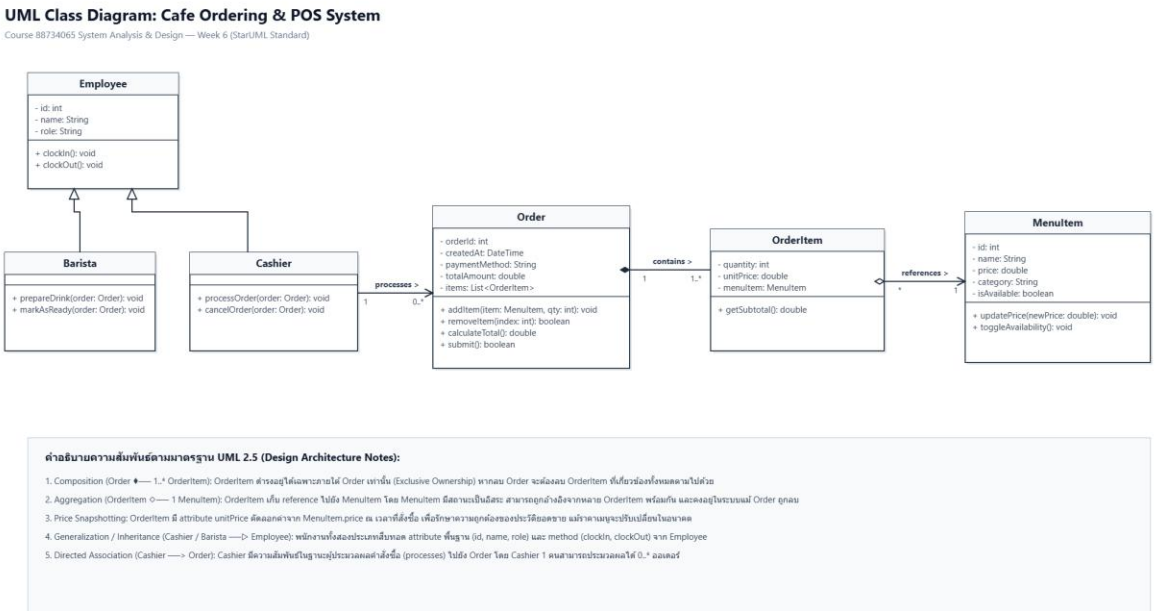
- **Acceptance Criteria:** Given อยู่หน้าจัดการเมนู, When ปรับสต็อกหรือแก้ไขราคา, Then ระบบบันทึกข้อมูลและอัปเดตไปยังหน้าจอ POS ประจำสาขานั้นทันที

**US-05: สรุปรายงานยอดขายเรียลไทม์:** ในฐานะเจ้าของร้าน
ฉันต้องการดูรายงานยอดขายรวมและเมนูขายดีประจำสาขาแบบเรียลไทม์ เพื่อวางแผนการดำเนินงาน

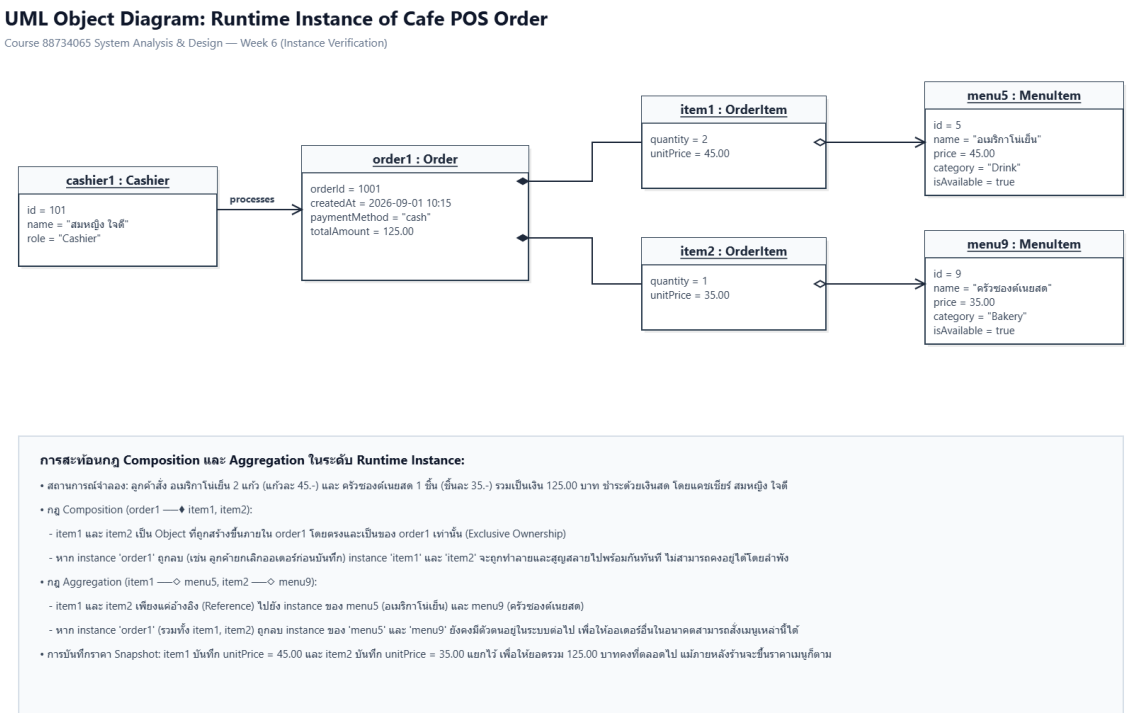
- **Acceptance Criteria:** Given มีออเดอร์ใหม่เกิดขึ้น, When เปิดหน้ารายงานยอดขาย, Then แดชบอร์ดสรุปยอดขายรวม จำนวนออเดอร์ และเมนูขายดีจะอัปเดตทันทีโดยไม่ต้องรีเฟรช

## 5. การออกแบบเชิงวัตถุ (Object-Oriented Analysis & Design - OOAD)

**5.1 แผนภาพคลาสและอ็อบเจกต์:** ระบบออกแบบ Domain Models ตามหลักการ OOAD โดยมีคลาสหลัก 3 คลาส ได้แก่ Order, OrderItem, และ MenuItem เชื่อมโยงกันตามความสัมพันธ์เชิงวัตถุ




*ภาพที่: Class Diagram ระบบร้าน Minions Cafe*

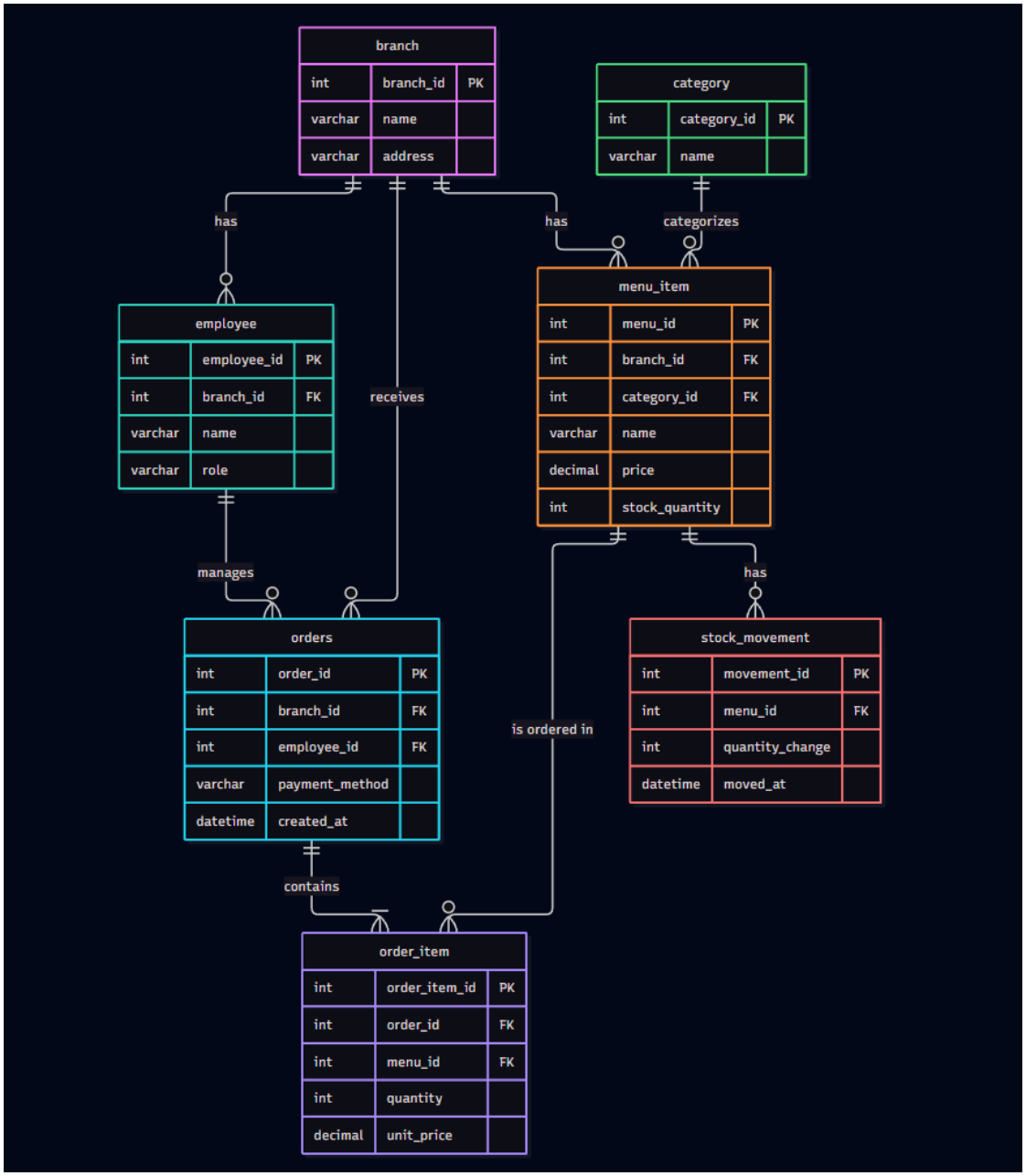



*ภาพที่: Object Diagram แสดงสถานะอ็อบเจกต์ขณะทำรายการขายจริง*

**5.2 การออกแบบตามหลัก 3NF ในระดับคลาส:** ในคลาส Order และตาราง orders จริงในฐานข้อมูล จะไม่มีการเก็บฟิลด์ total_amount เนื่องจากยอดรวมถือเป็น Derived Attribute ตามหลัก Third Normal Form (3NF) โดยคำนวณสดผ่านเมธอด getSubtotal() ของ OrderItem (quantity * unit_price) ช่วยป้องกันปัญหา Data Inconsistency ได้อย่างเด็ดขาด

# 6. การออกแบบฐานข้อมูล (Database Design)

**6.1 โครงสร้างฐานข้อมูลเชิงสัมพันธ์:** โครงสร้างฐานข้อมูลเชิงสัมพันธ์ประกอบด้วย 7 ตารางหลักที่เชื่อมโยงกันด้วย Foreign Key รองรับระบบหลายสาขาผ่านคอลัมน์ branch_id และมีตาราง stock_movement สำหรับเก็บบันทึก Audit Trail




*ภาพที่: Entity-Relationship Diagram (ER Diagram 7 ตาราง)*

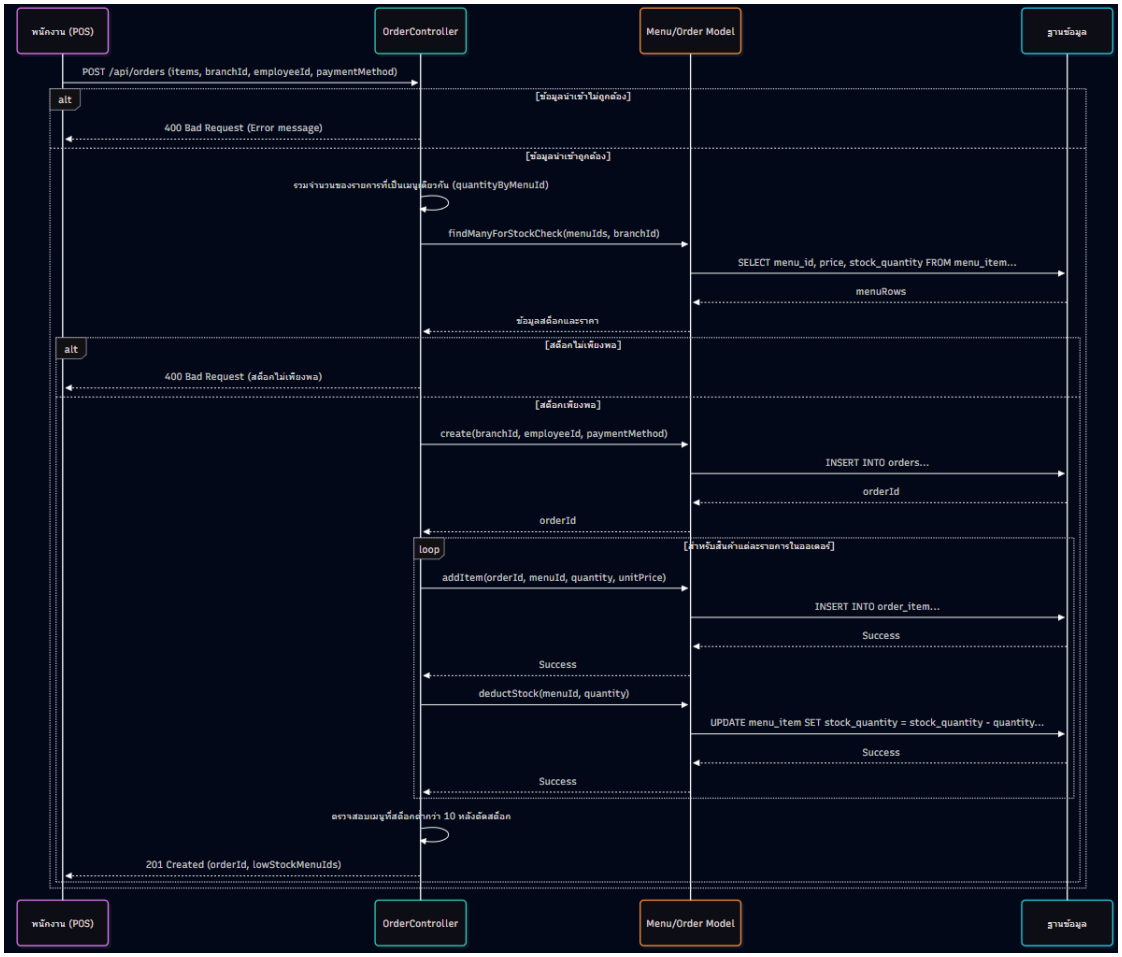
| ชื่อตาราง | Primary Key | Foreign Keys & Attributes | หน้าที่ความรับผิดชอบ |
|---|---|---|---|
| branch | branch_id (PK) | name, address | เก็บข้อมูลสาขา (สาขา 1 ม.บูรพา, สาขา 2 แหลมทอง) |
| category | category_id (PK) | name | หมวดหมู่สินค้า (1: กาแฟ, 2: ชา, 3: เบเกอรี่) |
| employee | employee_id (PK) | branch_id (FK), name, role | พนักงานประจำสาขา (สมชาย-Cashier, สมหญิง-Manager) |
| menu_item | menu_id (PK) | branch_id (FK), category_id (FK), name, price, stock_quantity | รายการเมนูและสต็อกคงเหลือแยกตามสาขา |
| orders | order_id (PK) | branch_id (FK), employee_id (FK), payment_method, created_at | หัวบิลคำสั่งซื้อ (ไม่มี total_amount ตามหลัก 3NF) |
| order_item | order_item_id (PK) | order_id (FK), menu_id (FK), quantity, unit_price | รายการสินค้าในคำสั่งซื้อ (คำนวณราคาย่อยได้) |
| stock_movement | movement_id (PK) | menu_id (FK), quantity_change, moved_at | บันทึกประวัติการตัดสต็อกเข้า-ออกตามแกนเวลา |

# 7. การออกแบบกระบวนการและพฤติกรรมระบบ (Process & Behavioral Design)

**7.1 Sequence Diagram:** กระบวนการสั่งซื้อและตัดสต็อก (Order-to-Stock)

ได้รับการออกแบบให้ทำงานแบบ Atomic Transaction

เพื่อป้องกันปัญหาความไม่สอดคล้องของข้อมูลเงินและสต็อก




*ภาพที่: Sequence Diagram แสดงขั้นตอนการสั่งซื้อ ตรวจสอบสต็อก และออกใบเสร็จ*

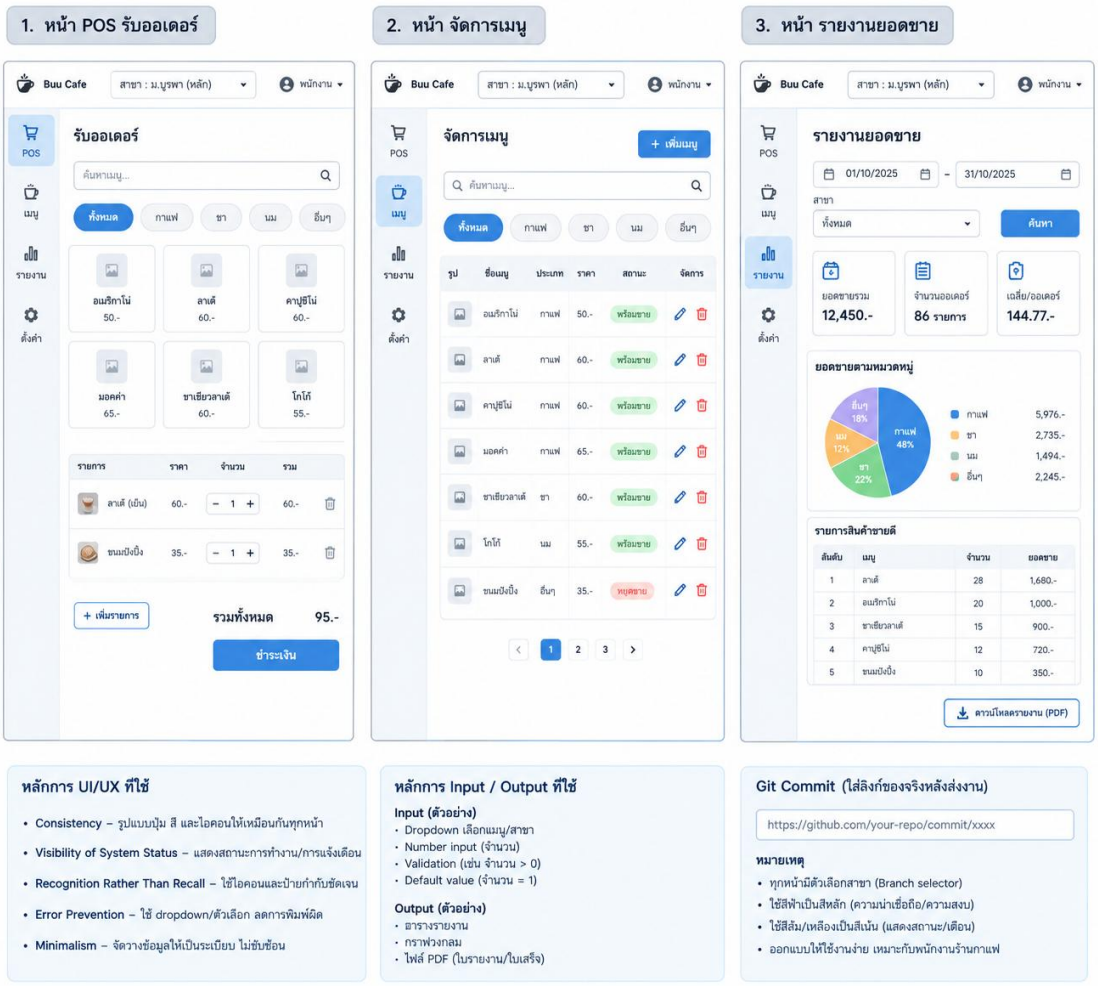
**7.2 ตรรกะทางธุรกิจในกระบวนการสั่งซื้อ:** เมื่อได้รับคำขอ POST /api/orders เซิร์ฟเวอร์จะทำ Pre-check ตรวจสอบสต็อกสินค้าทุกชิ้นในตะกร้าพร้อมจำลองการล็อกแถว (SELECT ... FOR UPDATE) หากมีรายการใดสต็อกไม่พอจะ Rollback ยกเลิกคำขอทันที หากสต็อกพอจะบันทึกออเดอร์ ตัดสต็อก บันทึกลง stock_movement และสร้างใบเสร็จส่งกลับสถานะ 201 Created

```
// POST /api/orders - ตรวจสอบสต็อกด้วยตรรกะ Row-Locking ก่อนตัดสต็อก (code/server.js)
for (const item of items) {
  const menuItem = menuItems.find(m => m.menu_id === parseInt(item.menuId) && m.branch_id ===
```

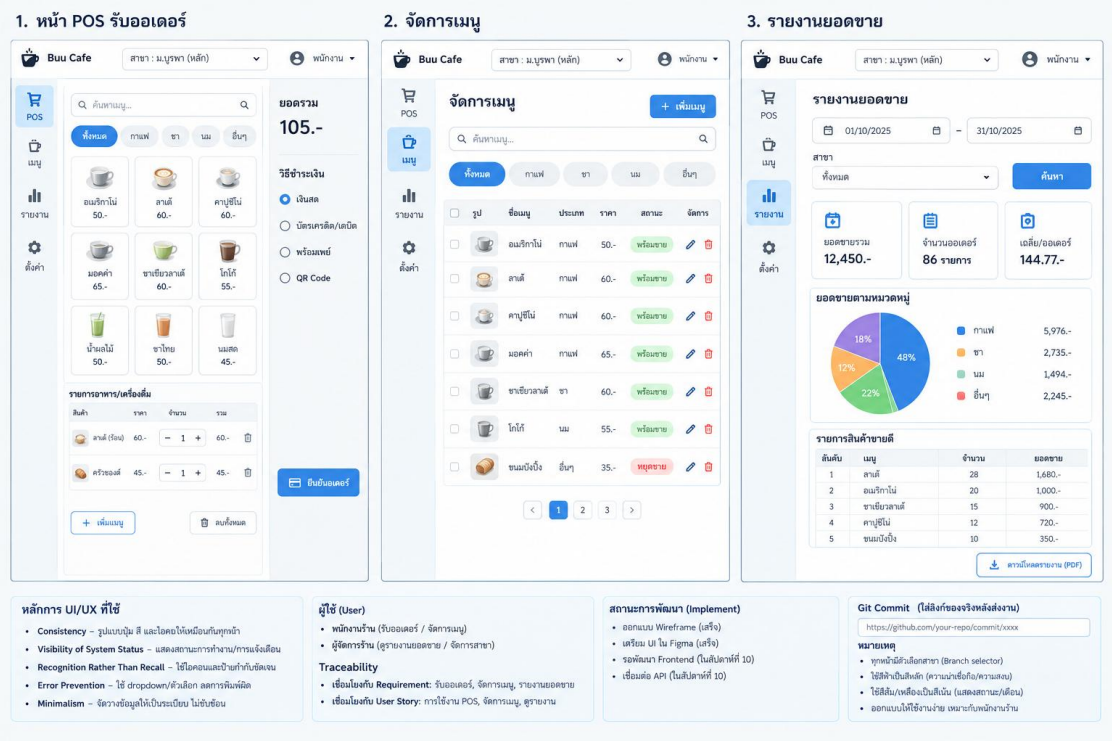
```
parsedBranchId);
  if (menuItem.stock_quantity < requestedQty) {
    return res.status(400).json({ error: `สินค้า ${menuItem.name} สต็อกไม่เพียงพอ` }); // Rollback
  }
  calculatedTotal += requestedQty * menuItem.price;
  orderItemsRecord.push({ menuItem, quantity: requestedQty, unitPrice: menuItem.price });
}
// บันทึกคำสั่งซื้อและตัดสต็อกสินค้าในคลังพร้อมกัน (Commit)
orders.push({ order_id: nextOrderId++, branch_id: parsedBranchId, total_amount: calculatedTotal });
for (const rec of orderItemsRecord) {
  rec.menuItem.stock_quantity -= rec.quantity; // ตัดสต็อกจริง
  stockMovements.push({ menu_id: rec.menuItem.menu_id, quantity_change: -rec.quantity });
}
```

# 8. การออกแบบ Input/Output และส่วนประสานผู้ใช้

**8.1 การออกแบบ Mid-Fi Wireframe:** หน้าจอระบบได้รับการออกแบบตามหลัก Usability โดยแบ่งเป็น 3 หน้าจอหลัก: POS รับออเดอร์, จัดการเมนู & สต็อก, และรายงานยอดขาย พร้อมตัวเลือกสาขา (Branch Selector) ที่มุมบนขวาทุกหน้าจอ




*ภาพที่: Mid-Fi Wireframe หน้าจอ POS, จัดการเมนู และรายงานยอดขาย*




*ภาพที่: Input / Output Design แสดงฟอร์มรับออเดอร์และใบเสร็จรับเงิน*

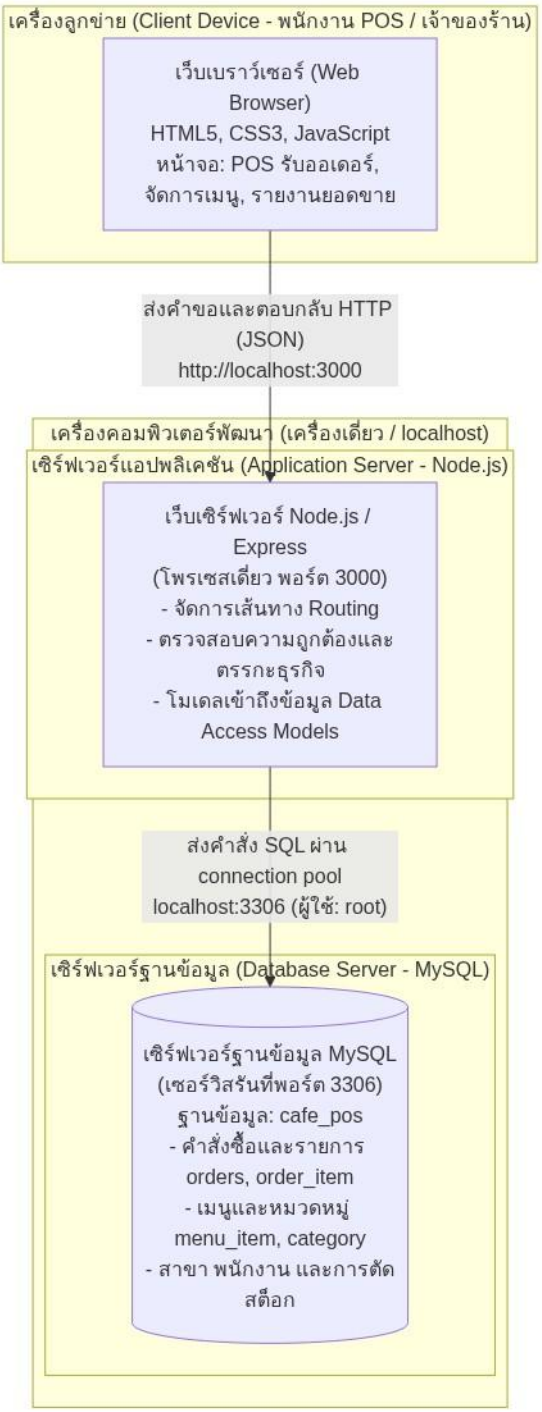
**8.2 รายละเอียดฟอร์มและใบเสร็จ:** หน้าจอ POS มีช่องค้นหาข้อความสด ปุ่มหมวดหมู่ ป้ายแจ้งเตือนสต็อก ตารางตะกร้าปรับจำนวนได้ และช่องคำนวณเงินสด/เงินทอนสด

ส่วนใบเสร็จรับเงินออกแบบเลียนแบบกระดาษความร้อน 80mm พร้อมปุ่มพิมพ์ใบเสร็จความร้อนผ่านคำสั่ง window.print() และหน้ารายงานสรุปยอดขายแสดง KPI 4 ใบ

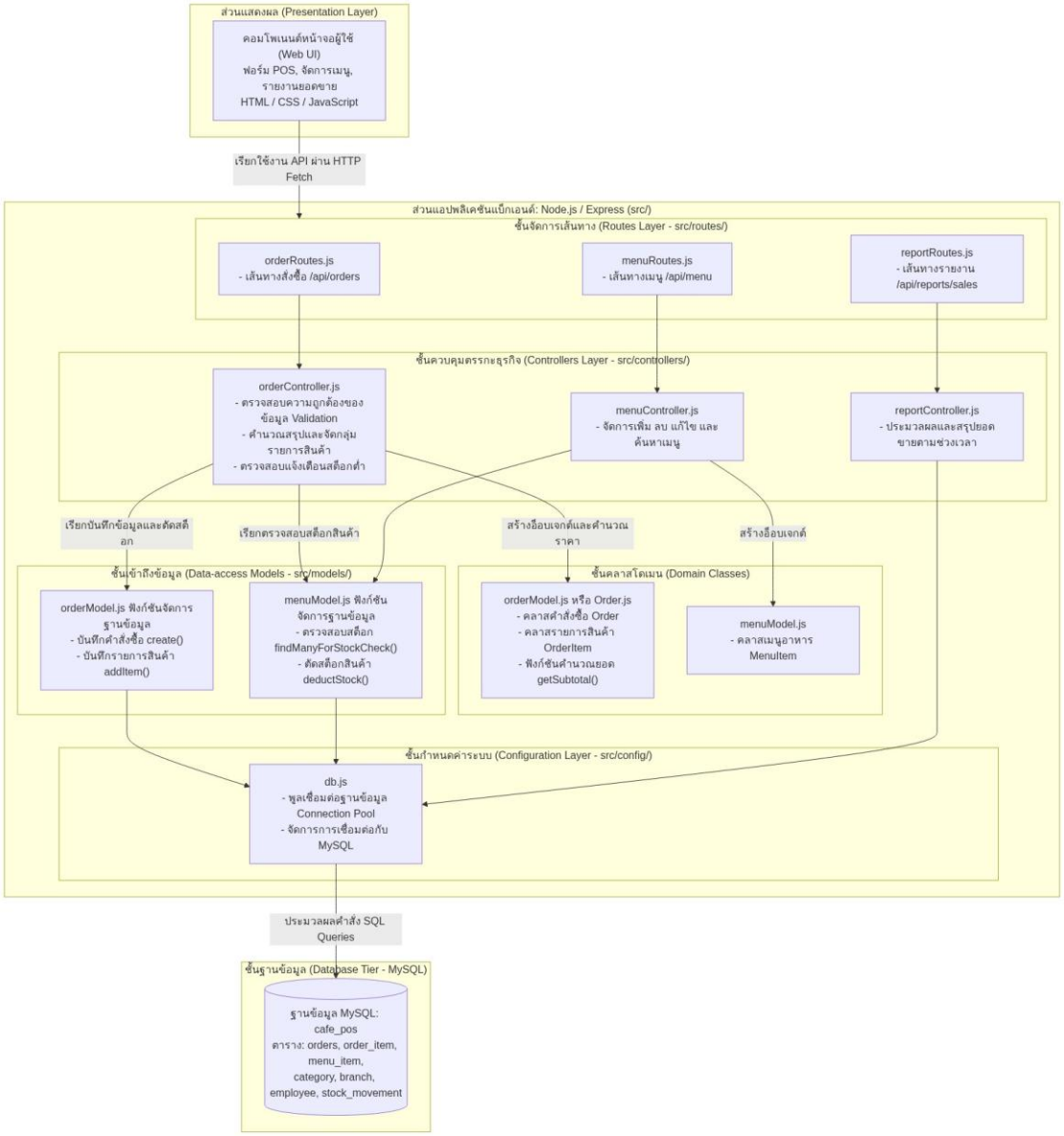
พร้อมตารางจัดอันดับเมนูขายดีและกราฟเปรียบเทียบ 2 สาขา

## 9. สถาปัตยกรรมระบบ (System Architecture - 4+1 View Model)

**9.1 ภาพรวมสถาปัตยกรรม:** ระบบ Minions Cafe เลือกใช้สถาปัตยกรรมแบบ Monolithic Architecture ที่จัดโครงสร้างภายในแบบ Layered Architecture (Monolithic + Layered) เนื่องจากทีมงานมี 5 คน และมีระยะเวลาพัฒนา 1 เทอม ช่วยลด Overhead ของ Distributed Services และรักษาความสอดคล้องของข้อมูลได้ง่ายกว่า Microservices




*ภาพที่: Physical View แสดงโหนดการติดตั้งบน localhost:3000 และพอร์ตฐานข้อมูล*




*ภาพที่: Development View แสดงการแบ่ง Layer ภายใน Repository จริง*

**9.2 ความสอดคล้องของสถาปัตยกรรม:** สถาปัตยกรรมภายในแบ่งเป็น Presentation Layer (HTML/CSS/JS), Routing & Controller Layer (server.js), Domain Models Layer (menuModel, orderModel) และ Data Store Layer สอดคล้องกับโครงสร้างโฟลเดอร์จริงใน Repository (Traceability)

## 10. ผลการทดสอบ Usability (Usability Testing)

**10.1 ผลการทดสอบ 3 ภารกิจ:** ในสัปดาห์ที่ 12 ทีมงานได้ทดสอบระบบจริงกับผู้ใช้ตัวแทนด้วยระเบียบวิธี Task-Based Testing ร่วมกับ Think-Aloud Protocol ครอบคลุม 3 ภารกิจหลัก:

| ภารกิจทดสอบ | สถานการณ์ทดสอบ | เวลาเฉลี่ย | ผลลัพธ์ | ปัญหาที่พบและการปรับแก้จริง |
|---|---|---|---|---|
| Task 1: Happy Path POS | สั่ง Latte 1 แก้ว จ่ายเงินสด และออกใบเสร็จ | 42 วินาที | สำเร็จ | ปุ่มยืนยันออเดอร์เดิมกลืนกับพื้นหลัง ⟶ ปรับแก้เป็นปุ่มสีเขียว Emerald เด่นชัด |
| Task 2: Out of Stock | พยายามสั่ง Espresso เกินสต็อกที่มีอยู่ 8 แก้ว | 35 วินาที | สำเร็จ | พบว่าเมนูสต็อก 0 ยังคลิกได้ ⟶ แก้ไขโค้ดเพิ่ม FR-11 ปิดปุ่มทันทีที่สต็อกเป็น 0 |
| Task 3: Sales Report | ตรวจสอบยอดขายรวมและเมนูขายดีหลังขายเสร็จ | 28 วินาที | สำเร็จ | ผู้ใช้พึงพอใจที่ข้อมูลสรุปอัปเดตแบบเรียลไทม์ทันทีโดยไม่ต้องกดรีเฟรชหน้าจอ |

**10.2 การปรับปรุงระบบจากผลการทดสอบ:** ทีมงานได้ดำเนินการแก้ไขปัญหาในระบบจริงครบทั้ง 4 ข้อ ได้แก่: 1. แก้ไข UI-02 (Critical) เพิ่ม FR-11 ปิดปุ่มเมนูสต็อก 0 ทันที 2. แก้ไข UI-01 (Major) ปรับปุ่มยืนยันออเดอร์เป็นสีเขียว Emerald เด่นชัด 3. แก้ไข UI-03 (Minor) เพิ่มช่องรับเงินสดและคำนวณเงินทอนสด และ 4. แก้ไข UI-04 (Minor) ปรับขนาดแถบเลือกสาขาให้ชัดเจนทุกหน้าจอ

## 11. ผลการรวมระบบ (System Integration)

**11.1 วงจรการทำงานที่ทดสอบครบถ้วน:** ในสัปดาห์ที่ 13 ได้ทำการทดสอบ End-to-End System Integration Flow ครบวงจร: Order -> Payment -> Stock Deduction -> Receipt -> Report

- **1. สั่งซื้อ (Order):** พนักงานเลือกสาขา 1 สั่ง Latte (45.-) + Espresso (35.-) + ครัวซองต์ (55.-) ยอดในตะกร้าคำนวณสดทันทีเป็น 135.00 บาท
- **2. ชำระเงิน (Payment):** เลือกชำระเงินสด กรอกรับเงินมา 200.00 บาท ระบบคำนวณเงินทอน 65.00 บาท แสดงผลสดทันที
- **3. ตัดสต็อก (Stock Deduction):** กดยืนยันออเดอร์ ระบบส่งคำขอ POST /api/orders ตัดสต็อกในระบบจริงทันที (Latte เหลือ 11, Espresso เหลือ 7, ครัวซองต์ เหลือ 8)
- **4. ออกใบเสร็จ (Receipt):** หน้าต่างใบเสร็จรับเงินเด้งขึ้นมาทันที แสดงเลขที่ ORD-20261030-105 วันเวลา สาขา พนักงาน และยอดรวมถูกต้อง 100%
- **5. สะท้อนยอดในรายงาน (Report):** เมื่อเปิดแท็บ 'รายงานยอดขาย' ยอดขายรวมสุทธิเพิ่มขึ้น 135 บาททันที และออเดอร์ใหม่ปรากฏในประวัติทันทีโดยไม่ต้องรีโหลดหน้าเว็บ

**11.2 ตารางสรุปผลการทดสอบการรวมระบบ (End-to-End Integration Test Matrix)**

| รหัสทดสอบ | กรณีทดสอบ | ขั้นตอนการทดสอบ | ผลลัพธ์ที่เกิดขึ้นจริง | สถานะ |
|---|---|---|---|---|
| TC-INT-01 | แสดงเมนูตามสาขา | เลือกสาขา 1 (ม.บูรพา) | แสดงเฉพาะเมนู 12 รายการของสาขา 1 | ผ่าน (Pass) |
| TC-INT-02 | คำนวณยอดเงินตะกร้าสด | เลือก Latte + Espresso + ครัวซองต์ | ยอดรวมคำนวณสดเป็น ฿135.00 | ผ่าน (Pass) |
| TC-INT-03 | คำนวณเงินสดและเงินทอน | เลือกเงินสด รับเงิน 200.00 บาท | แสดงยอดเงินทอน ฿65.00 สด | ผ่าน (Pass) |
| TC-INT-04 | ยืนยันออเดอร์และตัดสต็อก | กดยืนยันออเดอร์ POST /api/orders | สต็อกลดลงตรงตามจำนวนสั่งซื้อจริง | ผ่าน (Pass) |

| | | | | |
|---|---|---|---|---|
| TC-INT-05 | ออกใบเสร็จความร้อน | เปิด Thermal Receipt Modal | แสดงบิล ORD-20261030-105 ครบถ้วน | ผ่าน (Pass) |
| TC-INT-06 | สะท้อนยอดรายงานสด | เปิดแท็บ 'รายงานยอดขาย' | ยอดขายเพิ่ม 135 บาททันทีไม่ต้องรีเฟรช | ผ่าน (Pass) |
| TC-INT-07 | ป้องกันสินค้าหมดสต็อก | สั่งซื้อจนสต็อกเป็น 0 ชิ้น | ปุ่ม [+] ถูก disable ขึ้นป้าย 'สินค้าหมด' | ผ่าน (Pass) |

## 12. ผลการตรวจสอบคุณภาพและความปลอดภัยของซอฟต์แวร์

**12.1 การตรวจสอบเทียบ Checklist:** ในสัปดาห์ที่ 14 ทีมงานได้ตรวจสอบโค้ดจริงเทียบกับ Code Review Checklist 5 ข้อสำหรับระบบ CRUD:

| Endpoint | Method | Input Valid. | IDOR | SQL Injection | Error Hand. | Race Condition | สรุปผล |
|---|---|---|---|---|---|---|---|
| /api/menu | GET | ผ่าน | ผ่าน (branch_id) | ผ่าน (?) | ผ่าน | ไม่เกี่ยวข้อง | ผ่าน |
| /api/menu/:id | PUT | ผ่าน | ผ่าน (แก้ไขแล้ว) | ผ่าน (?) | ผ่าน | ไม่เกี่ยวข้อง | ผ่าน |
| /api/orders | POST | ผ่าน | ผ่าน (branch_id) | ผ่าน (?) | ผ่าน | ผ่าน (แก้ไขแล้ว) | ผ่าน |
| /api/reports/sales | GET | ผ่าน | ผ่าน (branch_id) | ผ่าน (?) | ผ่าน | ไม่เกี่ยวข้อง | ผ่าน |

**12.2 การแก้ไขความเสี่ยงสำคัญ:** ตรวจพบความเสี่ยงระดับ Critical เรื่อง Race Condition ในการอ่านและตัดสต็อกเมื่อมีคำสั่งซื้อพร้อมกัน ทีมงานได้แก้ไขจริงโดยนำ Database Transaction ร่วมกับ

SELECT ... FOR UPDATE มาใช้ใน server.js เพื่อล็อกแถวสต็อก ป้องกันสต็อกติดลบได้อย่างเด็ดขาด และแก้ไขปัญหา IDOR ใน PUT /api/menu/:id ด้วยการตรวจสอบสิทธิ์ความเป็นเจ้าของ branch_id

**12.4 แผนการจัดการความเสี่ยงที่เหลืออยู่ (Remaining Risks & Action Plan)**

| ข้อสังเกต / ความเสี่ยง | ระดับ | ผลกระทบ | แผนงานและแนวทางป้องกัน (Mitigation) |
|---|---|---|---|
| 1. การจำกัดอัตราคำขอ (Rate Limiting) | Medium | ป้องกัน DoS และคำสั่งซื้อซ้ำซ้อน | ติดตั้ง express-rate-limit จำกัด 60 req/min |
| 2. Audit Log ผู้ปรับปรุงสต็อกฉุกเฉิน | Low | ตรวจสอบย้อนหลังการแก้ไขสต็อกด่วน | เพิ่ม employee_id ในตาราง stock_movement |
| 3. ความปลอดภัย HTTP Response Header | Low | ป้องกัน Web Vulnerabilities พื้นฐาน | ติดตั้ง helmet middleware ใน Express Server |

## 13. สรุปผลและข้อเสนอแนะ (Conclusions & Recommendations)

**13.1 สรุปผลสัมฤทธิ์และบทเรียน:** โครงงาน Minions Cafe บรรลุวัตถุประสงค์ครบถ้วนทุกประการ สามารถทำงานได้อย่างถูกต้องทั้งในรูปแบบ Node.js Express Server และ Standalone Client Mode บทเรียนสำคัญคือการออกแบบที่มี Traceability เชื่อมโยง Requirement, Diagram และโค้ดจริง ช่วยลดข้อผิดพลาดในการพัฒนาย่างมาก

**13.2 ข้อจำกัดของระบบ:** ข้อจำกัดปัจจุบันคือระบบเซิร์ฟเวอร์ Express ใช้ In-memory Store สำหรับเก็บข้อมูลตัวอย่างเพื่อความสะดวกรวดเร็วในการนำเสนอ และหน้าต่าง QR PromptPay เป็นรูปแบบจำลอง (Simulation Modal)

**13.3 แนวทางพัฒนาต่อยอด:** แนวทางพัฒนาในอนาคตประกอบด้วย: การเชื่อมต่อ MySQL 8.0 อย่างถาวรผ่าน Connection Pool, การเพิ่มระบบ Login ด้วย JWT และ bcrypt, การเชื่อมต่อ Payment Gateway จริง (Omise / GB Prime Pay), และการพัฒนาระบบสมาชิกสะสมแต้ม 10 แต้มแลกฟรี 1 แก้ว

## 14. ภาคผนวก (Appendix: AI Disclosure Forms)

**14.1 สรุปการเปิดเผยการใช้งาน AI:** เพื่อความโปร่งใสทางวิชาการ โครงงาน Minions Cafe ได้จัดทำแบบฟอร์มเปิดเผยการใช้งาน AI ครบถ้วนตลอดภาคการศึกษา โดยมีสาระสำคัญสรุปดังนี้:

| สัปดาห์ | เอกสารทางการ | ขอบเขตการใช้งาน | เครื่องมือ AI | การตรวจสอบโดยมนุษย์ |
|---|---|---|---|---|
| สัปดาห์ที่ 5 | docs/ai-disclosure/wk05-ai-disclosure.md | ออกแบบ Git Branching และสคริปต์ตรวจ Repo | Claude / Gemini | ทดลองสลับ Branch ใน Git จริง |
| สัปดาห์ที่ 9 | docs/ai-disclosure/wk09-ai-disclosure.md | ช่วยร่าง API Spec และ Sequence Diagram | Claude 3.5 Sonnet | ตรวจสอบ Data Types เทียบ Use Case |
| สัปดาห์ที่ 10 | docs/ai-disclosure/wk10-ai-disclosure.md | จัดรูปแบบตาราง Input/Output และ SRS | Claude 3.5 Sonnet | ตรวจสอบฟิลด์เทียบกับ ER Diagram |
| สัปดาห์ที่ 13 | docs/ai-disclosure/wk13-ai-disclosure.md | ช่วยร่าง Test Scenarios สำหรับ E2E Test | Claude 3.5 Sonnet | ทดสอบกดสั่งซื้อจริงบนเว็บเบราว์เซอร์ |